

Percona Operator for PostgreSQL

Day-2 Operations Failure Runbook

Streaming-replication PostgreSQL pods on PVC storage, pgBackRest full/delta backups on a dedicated repo pod, and a self-hosted Prometheus / Grafana / Alertmanager stack driven by CronJobs.

A field guide to the failure modes that actually page you — what they look like, how to confirm the root cause with exact commands, how to recover safely, and how to stop them recurring. Grounded in Percona / Crunchy PGO v2.x behaviour (current line 2.9.x, PostgreSQL 15/16/17/18), Patroni HA, and pgBackRest.

Field	Value
Scope	Percona Operator for PostgreSQL v2.x (PerconaPGCluster CR)
HA layer	Patroni + Kubernetes DCS (Endpoints/ConfigMaps)
Backup	pgBackRest — repo-host pod, full + differential
Storage	PVC per instance (pgdata) + optional separate WAL PVC
Monitoring	Self-hosted Prometheus, Grafana, Alertmanager + CronJobs
Audience	Operations DBA / Platform SME on-call

How to use this runbook

Every scenario follows the same four-beat structure so you can move fast under pressure: **Symptoms** (what the alert or user reports), **Diagnosis** (the exact commands that confirm or rule out the cause), **Root Causes**, and **Resolution** with a **Prevention** note. Commands assume your cluster CR is named **cluster1** in namespace **pgo** — substitute your own. Instance pods are **cluster1-instance1-<suffix>-0**, the repo pod is **cluster1-repo-host-0**, and the pooler is **cluster1-pgbouncer-<suffix>**.

First-response toolkit — run these before anything else

Nine times out of ten these five commands localise the problem to a domain (HA, replication, storage, backup, or scheduling) within a minute.

```
# 1. Operator's own view of the cluster — is the CR Ready?
kubectl get pg -n pgo
kubectl get pg cluster1 -n pgo -o yaml | grep -A25 "status:"

# 2. Every pod, its phase, restarts, and node
kubectl get pods -n pgo -o wide -l postgres-operator.crunchydata.com/cluster=cluster1

# 3. Patroni's truth about who is primary and replica lag
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list

# 4. pgBackRest health: stanza status + WAL archive min/max
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest info

# 5. Recent operator decisions (why it did / didn't act)
kubectl logs deploy/percona-postgresql-operator -n pgo --tail=120
```

Fig 0.1 — the five commands that start every investigation.

READ THE CR STATUS FIRST

The Operator distinguishes an **operator error** (reconcile failure) from an **unschedulable pod**. If a pod cannot be scheduled the cluster shows **initializing**, not **error** — so a cluster stuck in *initializing* for a long time is your cue to read the operator logs, not to assume Postgres itself is broken.

Three switches change the Operator's behaviour and appear throughout this runbook — know them cold before you touch a production cluster:

Switch	Effect	When to use
spec.pause: true	Gracefully stops all pods (primary, replicas, pgBackRest, jobs); data volumes retained	Node maintenance, emergency stop, cost pause
spec.unmanaged: true	Operator stops reconciling entirely; probes still run; your manual changes survive	Manual surgery on Patroni/PG config
patronictl edit-config	Change Patroni-managed PG params via DCS at runtime	Tuning without a CR round-trip (temporary)

THE OPERATOR WILL FIGHT YOU

If you edit any object the Operator owns (StatefulSet, ConfigMap, Service, PG parameters it manages) while the cluster is **managed**, the next reconcile reverts your change. For hands-on recovery set **spec.unmanaged: true** first, do the work, then hand control back. Do not leave a cluster unmanaged permanently — health-check probes still fire and you lose self-healing.

Scenario index

A. High Availability, Patroni & Failover

- A1 Primary down but no failover happens
- A2 Split-brain — two pods think they are primary
- A3 Failover / restart flapping (probes & resource starvation)
- A4 Switchover hangs — Operator stuck in ChangePrimaryAndWait
- A5 Replica stuck 'creating replica' / won't rejoin

B. Streaming Replication

- B1 Replica lag growing steadily
- B2 Replication broken — slot inactive, wal_status = lost
- B3 Replication-slot bloat silently filling the disk
- B4 Synchronous replication stalls all writes
- B5 Timeline divergence after an unclean failover

C. Storage & PVC

- C1 Data PVC full — CrashLoop or read-only primary
- C2 WAL volume full — primary halts
- C3 PVC stuck Pending (zone / affinity / StorageClass)
- C4 PVC will not expand
- C5 Pod stuck ContainerCreating — volume multi-attach after node loss
- C6 Orphaned PV / reclaim-policy data-loss trap

D. pgBackRest & Backups

- D1 Stanza not created / config hash mismatch
- D2 WAL archiving failing — the pg_wal fill cascade
- D3 Full / delta backup job stuck or failing
- D4 Repo storage full / retention not pruning
- D5 Restore & PITR failures
- D6 Repo-host pod down — archive-push failing fleet-wide

E. Pods, Operator & Lifecycle

- E1 Cluster stuck 'initializing' / pods Pending
- E2 Database pod CrashLoopBackOff
- E3 OOMKilled — memory sizing
- E4 Operator reconcile errors / stuck reconciliation
- E5 Rolling (smart) update stuck mid-rollout
- E6 Config change silently reverted by the Operator

F. Connections & PgBouncer

- F1 Too many connections / pool exhaustion
- F2 PgBouncer pod failing or auth errors
- F3 App writes hitting a replica after failover (stale routing)

G. Monitoring, Alerting & CronJobs

- G1 Exporter down / no PostgreSQL metrics

G2 Prometheus target flaps after every failover

G3 Alertmanager not firing (or firing everything)

G4 Backup / monitoring CronJobs silently not running

A. High Availability, Patroni & Failover

In the Operator, Patroni — not Kubernetes — owns the primary/replica decision. Patroni stores cluster state in a **Distributed Configuration Store** (in Kubernetes that is Endpoints/ConfigMaps, not etcd), holds a **leader lease** that the primary must renew, and exposes a REST API on port **8008** that backs the pods' liveness/readiness probes. Almost every HA surprise traces back to one of those three things: the DCS, the lease, or the probes.

Scenario A1 Primary down but no failover happens

Symptoms

- Primary pod is Down/NotReady yet no replica is promoted; cluster read-only
- `patronictl list` shows no Leader, or a Leader in a bad state
- Applications time out on writes; the `-ha` service has no ready endpoint

Diagnosis

Failover requires a healthy replica *and* a reachable DCS. Confirm what Patroni sees, then confirm the DCS is writable.

```
# What does each Patroni node believe?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list
# Look for: is any node 'Leader'? are replicas 'streaming' or 'in archive recovery'?

# Is the DCS reachable? Patroni logs will scream if it is not
kubectl logs cluster1-instance1-abcd-0 -n pgo -c database | grep -iE "dcs|lock|leader|ttl"

# Are the replicas actually eligible? A replica far behind won't be promoted
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl list -e
```

WARNING

No failover with a single healthy node — if the only surviving pod is the failed primary, or every replica is lagging past the safety threshold, Patroni correctly refuses to promote. That is data-loss protection, not a bug.

ROOT CAUSES

DCS unreachable — Patroni cannot acquire/renew the leader lease, so it will not promote (loss of the lock demotes, it never blindly promotes).

No eligible replica: all replicas are down, lagging, or still bootstrapping.

`failover_priority`: 0 or a paused Patroni (`patronictl pause`) suppressing automatic action.

Quorum/synchronous mode requiring an acknowledgement no surviving node can give.

Resolution

First restore the DCS or replica eligibility; only force a promotion as a last resort and with eyes open about data loss.

```
# Is Patroni paused (maintenance mode)? If so, resume it.
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl resume

# If a good replica exists but wasn't chosen, trigger a controlled failover
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl failover --candidate cluster1-instance1-wxyz-0 --force

# LAST RESORT: no healthy peer, primary volume intact -> promote in place by
# letting the Operator restart the instance after fixing the underlying fault.
```

Fig A1 — prefer resume / controlled failover over any forced promotion.

DANGER

Never hand-promote two nodes. If you bypass Patroni with a raw `pg_ctl promote` while the old primary can still come back, you have manufactured split-brain (Scenario A2).

PREVENTION

Alert on **leader-lease age** and DCS reachability, not just pod readiness.

Keep at least one replica inside the lag threshold at all times (Scenario B1).

Run periodic game-day failovers so you trust the automation before you need it.

Scenario A2 Split-brain — two pods think they are primary

Symptoms

- Two nodes report Leader / accept writes
- Diverging row counts between pods; unique-key violations on the standby after re-sync
- Patroni logs mention a lost lock, a timeline bump, or 'promoted self'

Diagnosis

Split-brain is the one to treat as an incident, not a ticket. Establish which node holds the real lease and which one wrote data it should not have.

```
# Which node currently holds the lock, and on what timeline?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list

# Compare timelines and last write LSN on each candidate
kubectl exec -it <podA> -n pgo -c database -- \
psql -tAc "SELECT pg_is_in_recovery(), pg_current_wal_lsn(), timeline_id
FROM pg_control_checkpoint();"
kubectl exec -it <podB> -n pgo -c database -- \
psql -tAc "SELECT pg_is_in_recovery(), pg_current_wal_lsn(), timeline_id
FROM pg_control_checkpoint();"
```

ROOT CAUSES

A network partition let a replica lose sight of the DCS, time out the old leader, and promote itself while the original primary kept serving on the other side.

Someone promoted a node manually (or via `pg_ctl`) outside Patroni.

DCS TTL too short for the environment's network jitter, causing spurious lock loss.

Storage that allowed the same PVC to attach in two places (see C5).

Resolution

Pick the survivor, fence the loser, and rebuild it from the survivor. There is no merge — you choose which write history lives.

```
# 1. Freeze the cluster so the Operator doesn't reconcile mid-surgery
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":true}}'

# 2. Decide the survivor (usually the node that held the lease + more committed data).
# Stop PostgreSQL on the loser immediately to end the divergent writes.
kubectl exec -it <loser-pod> -n pgo -c database -- patronictl pause
kubectl exec -it <loser-pod> -n pgo -c database -- pg_ctl -D /pgdata/pg17 stop -m fast

# 3. Reinitialise the loser from the survivor (discards its divergent WAL)
kubectl exec -it <survivor-pod> -n pgo -c database -- \
patronictl reinit cluster1-ha <loser-pod>

# 4. Resume and hand control back once patronictl list is clean
kubectl exec -it <survivor-pod> -n pgo -c database -- patronictl resume
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":false}}'
```

Fig A2 — fence, choose, rebuild. `reinit` throws away the loser's history.

PREVENTION

Use an odd DCS member count and tune the Patroni `TTL` / `loop_wait` / `retry_timeout` to your real network latency.

Enable synchronous replication for zero-data-loss clusters so a partitioned replica cannot silently diverge.

Forbid manual promotion in runbooks; the only promotion path is `patronictl`.

Scenario A3 Failover / restart flapping (probes & resource starvation)

Symptoms

- Pods cycle Ready -> NotReady -> Ready; periodic unplanned failovers
- Liveness probe on `:8008` failing intermittently
- OOMKilled or CPU-throttled events on the database container

Diagnosis

Flapping is usually the platform starving Patroni's health endpoint, not Postgres dying. Correlate probe failures with resource pressure.

```
# Probe config + recent restart reasons
kubectl describe pod cluster1-instance1-abcd-0 -n pgo | \
grep -A3 -iE "liveness|readiness|last state|reason|oomkilled"

# Live resource pressure
kubectl top pod -n pgo -l postgres-operator.crunchydata.com/cluster=cluster1

# Patroni REST health as Kubernetes sees it
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
curl -sk https://localhost:8008/health
```

ROOT CAUSES

Memory limit too low -> OOMKill of Postgres or the Patroni sidecar, tripping the probe.

CPU limit throttling the Patroni loop so lease renewal misses its window.

Aggressive probe timings (short period/timeout) versus a busy node.

Underlying node instability (kubelet pressure, disk IO saturation).

Resolution

Right-size resources and, only if needed, relax probe timing — do not just widen probes to mask an OOM.

```
# Give the DB instance headroom (values illustrative — size to your workload)
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"instances": [{"name": "instance1",
"resources": {"requests": {"cpu": "2", "memory": "8Gi"},
"limits": {"memory": "8Gi"}}}]}'
# Keep memory request == limit for the DB so it lands in Guaranteed QoS.
```

Fig A3 — Guaranteed QoS keeps the DB from being the first eviction target.

PREVENTION

Set memory request = limit on the database container (Guaranteed QoS).

Alert on OOMKilled events and container restart counts, not only on pod readiness.

Leave probe defaults unless you have proven the node is healthy but slow.

Scenario A4 Switchover hangs — Operator stuck in ChangePrimaryAndWait

Symptoms

- A rollout / restart triggers a switchover that never completes
- Operator log parked in a switchover call; CR status 'Progressing' indefinitely
- Old primary restarts repeatedly during the attempt

Diagnosis

A known failure shape: the Operator issues a `patronictl switchover` and blocks waiting for a new primary that never stabilises.

```
kubectl logs deploy/percona-postgresql-operator -n pgo --tail=200 | \
grep -iE "switchover|ChangePrimary|primary"

# Cross-check Patroni's own state — is the switchover actually mid-flight?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list
```

ROOT CAUSES

The candidate could not fully promote (lag, IO stall, or config error) so the switchover neither completes nor rolls back cleanly.

A rollout restart annotation raced with an in-flight Patroni action.

DCS write latency stretching the wait past the Operator's expectation.

Resolution

Take the cluster unmanaged, resolve the switchover by hand with patronictl, then resume.

```
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":true}}'

# Drive the switchover yourself to a known-good candidate
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl switchover cluster1-ha --candidate cluster1-instance1-wxyz-0 --force

# Once patronictl list shows a single healthy Leader + streaming replicas:
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":false}}'
```

Fig A4 — resolve the switchover manually, then let the Operator resume.

PREVENTION

Avoid firing rollout restarts while another change is reconciling.

Prefer scheduled maintenance windows for restarts of the primary instance set.

Track the Operator's Progressing condition duration and alert if it stalls.

Scenario A5 Replica stuck 'creating replica' / won't rejoin

Symptoms

- A replica pod stays NotReady; Patroni shows it 'creating replica' or 'in archive recovery'
- Replica bootstrap loops; pgBackRest restore or pg_basebackup repeatedly restarts
- After a failover the demoted old primary cannot rejoin

Diagnosis

A replica joins by fetching a base image (pgBackRest restore or pg_basebackup) then streaming. Find which stage is failing.

```
kubectl logs <replica-pod> -n pgo -c database | \
grep -iE "creating replica|pg_basebackup|restore|rewind|bootstrap|FATAL"

# If it is demoted-primary rejoin, Patroni may need pg_rewind – check for divergence
kubectl exec -it <replica-pod> -n pgo -c database -- \
patronictl list -e
```

ROOT CAUSES

The demoted primary diverged from the new timeline and needs `pg_rewind`, but `rewind` is failing (`wal_log_hints/checksums` off, or missing WAL).

`pgBackRest` cannot serve the base backup (stanza/repo problem — see D-series).

Insufficient space on the replica PVC to lay down the base backup.

Network policy blocking replication traffic to the primary.

Resolution

Let Patroni rebuild it. `reinit` is the blunt, reliable hammer when `rewind` won't converge.

```
# Force a clean rebuild of the stuck member from the current primary/repo
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl reinit cluster1-ha <replica-pod>

# If the PVC is simply too small/corrupt, deleting the PVC+pod makes the Operator
# recreate the member and re-bootstrap from pgBackRest (DESTRUCTIVE to that replica only):
kubectl delete pod <replica-pod> -n pgo
kubectl delete pvc <replica-pgdata-pvc> -n pgo # only after confirming it's a replica!
```

Fig A5 — `reinit` first; PVC delete only when you're certain it's a replica.

PREVENTION

Keep `wal_log_hints` (or data checksums) on so `pg_rewind` can work — the Operator defaults are sane; don't override them off.

Size replica PVCs with headroom for a full base backup plus live WAL.

Validate NetworkPolicies allow pod-to-pod on the replication port.

B. Streaming Replication

Physical streaming replication is the backbone under Patroni. The recurring pain is always about WAL: a replica that falls behind, a replication slot that either protects a replica by retaining WAL (good) or bloats the disk when the replica is gone (bad), and the standby-first ordering discipline for replication-sensitive parameters.

Scenario B1 Replica lag growing steadily

Symptoms

- Grafana shows replication lag climbing (seconds -> minutes)
- 2.9+ marks a lagging standby cluster Unready with a **StandbyLagging** condition
- Read queries on replicas return stale data

Diagnosis

Quantify the lag in bytes and time, and locate the bottleneck (network, replay, or IO).

```
-- On the PRIMARY: per-replica send/write/flush/replay lag
SELECT client_addr, state, sent_lsn, replay_lsn,
pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_bytes,
write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

```
-- On the REPLICA: is it replay-bound or receive-bound?
SELECT pg_is_in_recovery(),
pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn(),
pg_wal_lsn_diff(pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn()) AS pending;
```

Fig B1 — *receive_lsn* far ahead of *replay_lsn* means the replica can't replay fast enough.

ROOT CAUSES

Replay bottleneck: a single startup process replaying heavy WAL (bulk load, VACUUM, index build) faster than the replica can apply — pending WAL grows on the replica.

Receive bottleneck: network saturation or an undersized replica node.

Long-running query on the replica blocking replay when `max_standby_streaming_delay` is high.

Storage IO on the replica slower than the primary's write rate.

Resolution

Treat the actual bottleneck. Do not blindly raise timeouts — that trades lag for query cancellations.

```
# Confirm no runaway query is holding replay on the replica
kubectl exec -it <replica-pod> -n pgo -c database -- psql -c \
"SELECT pid, now()-query_start AS dur, wait_event_type, left(query,60)
FROM pg_stat_activity WHERE state='active' ORDER BY dur DESC LIMIT 5;"

# If replay-bound and you accept staleness, enable hot_standby_feedback trade-offs
# and tune via the Operator (patched into Patroni config, applied standby-first):
kubectl exec -it <replica-pod> -n pgo -c database -- \
patronictl edit-config --set postgresql.parameters.max_standby_streaming_delay=30s
```

Fig B1b — edit-config pushes through the DCS to all members consistently.

PREVENTION

Dashboard both byte-lag and time-lag; alert before the StandbyLagging threshold.

Give replicas storage/CPU at least equal to the primary — replay is single-threaded.

Schedule bulk operations for low-traffic windows; consider **recovery_min_apply_delay** only deliberately.

Scenario B2 Replication broken — slot inactive, wal_status = lost

Symptoms

- A replica dropped off and cannot resume; pg_stat_replication no longer lists it
- wal_status = lost on its slot; primary already recycled the WAL it needed
- Replica logs: requested WAL segment already removed

Diagnosis

This is the classic 'raised a parameter on the primary only, standby restarted, slot went inactive, WAL recycled' incident. Confirm the slot state and whether recoverable WAL exists.

```
-- On the PRIMARY
SELECT slot_name, active, wal_status,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;
-- wal_status: reserved(ok) | extended | unreserved(warning) | lost(replica must rebuild)
```

Fig B2 — wal_status = lost means streaming resume is impossible; rebuild the replica.

ROOT CAUSES

A replication-sensitive parameter (max_connections, max_wal_senders, max_worker_processes, max_prepared_transactions, max_locks_per_transaction) was raised on the primary only; the standby refused to start until it matched, went inactive long enough that WAL was recycled past its restart_lsn.

max_slot_wal_keep_size too small (or WAL pressure) let the primary drop needed WAL.

Replica down for longer than the retained WAL window.

Resolution

If wal_status is lost, streaming cannot resume — rebuild the replica. Then fix the ordering discipline that caused it.

```
# Rebuild the broken member cleanly from the primary/repo
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl reinit cluster1-ha <broken-replica-pod>
```

ROOT CAUSE

Standby-first ordering: for the five replication-sensitive parameters, RAISE on the standby first, restart it, then the primary; LOWER on the primary first. Under the Operator, set these via the CR / patronictl edit-config so every member gets the change consistently through the DCS — never ALTER SYSTEM on one node by hand.

PREVENTION

Set these params via the Operator CR so all members move together — no single-node drift.

Keep **max_slot_wal_keep_size** generous enough to survive a replica's longest expected outage, but bounded so a dead slot can't fill the disk (see B3).

Alert on **wal_status != reserved** and on any inactive slot.

Scenario B3 Replication-slot bloat silently filling the disk

Symptoms

- pg_wal on the primary grows without bound; free space shrinking daily
- An inactive slot with a very old restart_lsn retaining gigabytes of WAL
- Eventually cascades into Scenario C2 (WAL volume full) and a halted primary

Diagnosis

A slot for a replica that no longer exists (deleted pod, abandoned standby cluster, dead logical consumer) pins WAL forever. Find the offender.

```
SELECT slot_name, slot_type, active, wal_status,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal,
age(xmin) AS xmin_age
FROM pg_replication_slots
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
```

Fig B3 — an inactive slot with large retained_wal is the disk-fill culprit.

ROOT CAUSES

A replica/standby cluster was removed but its physical slot was left behind.

A logical replication subscriber died and its slot keeps pinning WAL (and xmin, which also blocks VACUUM cleanup).

max_slot_wal_keep_size unset (-1), so a dead slot retains WAL indefinitely.

Resolution

Drop the genuinely orphaned slot — but be certain it belongs to nothing you still need. Under the Operator, Patroni manages its own physical slots; do not drop those.

```
-- Verify it is truly orphaned (no matching replica in patronictl list), THEN:
SELECT pg_drop_replication_slot('orphaned_slot_name');

-- Bound the blast radius going forward (applied via the Operator, all members):
-- max_slot_wal_keep_size = e.g. '64GB' (cap WAL a slot may pin)
```

Fig B3b — dropping a live Patroni-managed slot will break a healthy replica. Confirm first.

PREVENTION

Alert on retained_wal per slot and on inactive slots older than N minutes.

Set a sane **max_slot_wal_keep_size** so a dead consumer can't take the cluster down.

When you decommission a standby, remove its slot as part of the runbook.

Scenario B4 Synchronous replication stalls all writes

Symptoms

- All write transactions hang; app COMMITs never return
- Cluster configured with synchronous replication and a sync standby just went away
- pg_stat_activity shows sessions waiting on SyncRep

Diagnosis

In synchronous mode a COMMIT waits for a standby ack. If the required number of sync standbys can't ack, writes block by design.

```
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \
"SELECT wait_event, count(*) FROM pg_stat_activity
WHERE wait_event='SyncRep' GROUP BY 1;"

kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \
"SHOW synchronous_standby_names; SHOW synchronous_commit;"
```

ROOT CAUSES

The sole synchronous standby failed, and synchronous_standby_names still requires it.

num_sync (quorum) set higher than the number of currently healthy standbys.

A network partition isolated the standby that must acknowledge.

Resolution

Restore a healthy standby (preferred) or, to stop the bleeding, relax the sync requirement — accepting the durability trade-off consciously.

```
# Preferred: bring a standby back so the quorum is satisfiable (reinit if needed)
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl reinit cluster1-ha <sync-standby-pod>

# Emergency only: reduce required sync members via Patroni (durability trade-off!)
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl edit-config --set synchronous_mode=false
```

Fig B4 — disabling sync unblocks writes but drops the zero-data-loss guarantee.

PREVENTION

Use quorum-based sync (ANY N) with enough standbys that losing one is tolerable.

Let Patroni manage `synchronous_standby_names` — don't hand-edit it under the Operator.

Alert specifically on SyncRep waits so a stall is unambiguous, not 'DB slow'.

Scenario B5 Timeline divergence after an unclean failover

Symptoms

- After failover, a rejoining node logs 'requested timeline N does not match'
- Replica cannot follow the new primary; pgBackRest complains about timelines
- Multiple timeline history (.history) files in the archive

Diagnosis

A promotion bumps the timeline. Nodes and the backup repo must agree on it. Check the timelines across members and in pgBackRest.

```
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl list -e # TL column must match across Leader + streaming replicas

kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest info # note the current timeline in wal archive min/max
```

ROOT CAUSES

The old primary served writes on the old timeline after a new primary appeared, diverging history (often paired with A2).

A replica bootstrapped from a backup on an older timeline and can't fast-forward.

Archive gaps prevented a clean timeline switch.

Resolution

Rebuild diverged members from the current primary; ensure pgBackRest has the new timeline's WAL so future restores are consistent.

```
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl reinit cluster1-ha <diverged-pod>

# Verify archiving is healthy on the NEW timeline, then take a fresh full backup
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db check
```

Fig B5 — after any real failover, validate archiving and take a fresh full backup.

PREVENTION

Always take a fresh full backup after a genuine failover/promotion.

Keep enough archive retention to span at least one timeline switch.

Prevent the split-brain that usually precedes divergence (A2 preventions).

C. Storage & PVC

Databases are the hardest thing to run on Kubernetes precisely because of storage. The Operator provisions a PVC per instance for **pgdata** and, if you configured it, a separate PVC for **pg_wal**. Almost every storage incident is one of: the volume filled, the volume can't attach, or the volume can't grow.

Scenario C1 Data PVC full — CrashLoop or read-only primary

Symptoms

- Postgres logs: 'could not extend file ... No space left on device'
- Primary crashes and won't restart, or refuses writes
- PVC usage at/near 100%

Diagnosis

Confirm which volume is full and what is consuming it before you touch anything.

```
# Space per mounted volume inside the pod
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- df -h /pgdata /pgwal

# Biggest consumers under pgdata (base = data, pg_wal = WAL, log = logs)
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
du -h --max-depth=1 /pgdata | sort -rh | head
```

ROOT CAUSES

Organic data growth outpaced the PVC size.

WAL accumulation from a stuck archive_command or a bloated slot spilling into pgdata (if WAL is not on its own volume — see C2/D2/B3).

Bloat from dead tuples with autovacuum unable to keep up.

Log files or temp files filling the same volume.

Resolution

Free enough space to get Postgres writable, then expand the PVC. Don't delete anything under pgdata blindly.

```
# Safe, immediate wins: truncate/rotate logs (NOT data), remove temp if isolated
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
bash -c 'ls -sh /pgdata/*/log/ | tail' # inspect before removing

# Then expand the PVC (StorageClass must allow it — see C4)
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"instances": [{"name": "instance1",
"dataVolumeClaimSpec": {"resources": {"requests": {"storage": "200Gi"}}}}}]}'
```

Fig C1 — grow via the CR; the Operator drives the PVC resize.

PREVENTION

Alert at 75% and page at 85% on both pgdata and pg_wal PVCs.

Put **pg_wal on its own PVC** so a WAL spike can't take down data writes.

Track table/index bloat and autovacuum backlog proactively.

Scenario C2 WAL volume full — primary halts

Symptoms

- Primary stops accepting writes; logs point at pg_wal / archive status
- If WAL shares the data volume, this presents as C1; if separate, /pgwal is 100%
- Often the downstream effect of D2 (archiving stuck) or B3 (slot bloat)

Diagnosis

A full WAL volume is almost always a symptom. Postgres cannot recycle WAL that is still unarchived or still pinned by a slot. Find which.

```
# Is WAL piling up because archiving is failing?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
ls -l /pgdata/*/pg_wal/archive_status/*.ready 2>/dev/null | wc -l # >0 growing = stuck

kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -tAc \
"SELECT last_archived_wal, last_failed_wal, failed_count FROM pg_stat_archiver;"

# Or because a slot pins it? (cross-check Scenario B3)
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \
"SELECT slot_name,active,wal_status FROM pg_replication_slots;"
```

Fig C2 — a growing count of .ready files + rising failed_count = archiving is the cause.

ROOT CAUSES

archive_command failing (repo down, stanza broken, repo full) — WAL cannot be recycled until archived (D2).

An inactive replication slot pinning WAL (B3).

A checkpoint storm or huge transaction generating WAL faster than it can drain.

Resolution

Fix the drain (archiving or slot) — that lets Postgres recycle WAL and free space. Expanding the volume only buys time.

```
# If archiving is the cause, repair pgBackRest first (see D2), then confirm drain:
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db check

# Temporary headroom while you fix the real cause: grow the WAL PVC
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"instances": [{"name":"instance1",
"walVolumeClaimSpec": {"resources": {"requests": {"storage":"100Gi"}}}}]}'
```

Fig C2b — expand to survive; fix archiving/slot to actually recover.

PREVENTION

Never remove files from `pg_wal` by hand — you can render the cluster unrecoverable.

Separate WAL PVC + independent alerting on it.

Alert on `pg_stat_archiver.failed_count` increasing — the earliest warning of D2.

Scenario C3 PVC stuck Pending (zone / affinity / StorageClass)

Symptoms

- New/replacement instance pod stuck Pending; its PVC is Pending
- Events: 'waiting for a volume to be created' or 'no available zone/node'
- Common after a node loss when the pod reschedules to a different zone

Diagnosis

The scheduler can't place the pod where the volume can be provisioned/attached. Read the PVC and pod events together.

```
kubectl describe pvc <pending-pvc> -n pgo | grep -A10 Events
kubectl describe pod <pending-pod> -n pgo | grep -A10 Events
kubectl get storageclass
kubectl get pvc <pvc> -n pgo -o jsonpath='{.spec.storageClassName}{"\n"}'
```

ROOT CAUSES

Zonal block storage (EBS/PD): the PV lives in zone A but the pod was scheduled to zone B.

StorageClass with `WaitForFirstConsumer` plus node affinity that can't be satisfied.

No default StorageClass, or the named class doesn't exist / has no capacity.

PVC selector/label mismatch (common when reusing pre-labelled PVs).

Resolution

Align pod placement with the volume's topology. For zonal storage, pin the instance set to the volume's zone or use topology-aware provisioning.

```
# Constrain the instance set to the zone where its volume can bind
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"instances": [{"name": "instance1",
"affinity": {"nodeAffinity": {"requiredDuringSchedulingIgnoredDuringExecution": {
"nodeSelectorTerms": [{"matchExpressions": [
{"key": "topology.kubernetes.io/zone", "operator": "In", "values": ["us-east-1a"]}
]}]}]}]}'
```

Fig C3 — zonal storage needs the pod pinned to the volume's zone (or a topology-aware SC).

PREVENTION

Use a StorageClass with **volumeBindingMode: WaitForFirstConsumer** so the volume is created in the zone the pod actually lands in.

Spread instances across zones deliberately; understand your CSI's topology behaviour.

Keep a default StorageClass with capacity headroom.

Scenario C4 PVC will not expand

Symptoms

- You raised storage in the CR but the PVC/PV stays the old size
- PVC event: 'storageclass does not support volume expansion'
- Or PVC shows FileSystemResizePending and never completes

Diagnosis

Volume expansion needs three things aligned: the StorageClass allows it, the CSI driver supports it, and (sometimes) a pod restart finalises the filesystem grow.

```
kubectl get storageclass <sc> -o jsonpath='{.allowVolumeExpansion}{"\n"}' # must be true
kubectl describe pvc <pvc> -n pgo | grep -A6 -iE "condition|resize|capacity"
kubectl get pvc <pvc> -n pgo -o jsonpath='{.status.capacity.storage} / requested {spec.resources.requests.storage}{"\n"}'
```

ROOT CAUSES

StorageClass has allowVolumeExpansion: false (or unset).

CSI driver/version doesn't support online expansion.

Shrinking is not supported — you requested a smaller size than current.

Filesystem resize pending a pod restart.

Resolution

Enable expansion on the class (for future volumes), then complete the pending resize; some drivers finalise only after the mounting pod restarts.

```
# Enable expansion on the StorageClass (affects new PVCs; existing may need driver support)
kubectl patch storageclass <sc> --type merge -p '{"allowVolumeExpansion":true}'

# If FileSystemResizePending, a controlled pod restart finalises the grow
kubectl delete pod <instance-pod> -n pgo # StatefulSet recreates it; PVC persists
```

Fig C4 — you cannot retro-enable expansion on a volume whose class/driver never allowed it.

PREVENTION

Choose an expansion-capable StorageClass from day one for all DB volumes.

Never provision DB PVCs at their maximum expected size 'to be safe' — size to grow.

Test an expansion in non-prod so you know whether your CSI needs the restart step.

Scenario C5 Pod stuck ContainerCreating — volume multi-attach after node loss

Symptoms

- After a node crash the pod reschedules but hangs in ContainerCreating
- Event: 'Multi-Attach error for volume ... already exclusively attached to one node'
- Seen with RWO block storage on spot/preemptible nodes

Diagnosis

RWO volumes attach to exactly one node. If the old node died without cleanly detaching, the new pod can't attach until the volume is released.

```
kubectl describe pod <pod> -n pgo | grep -A6 -iE "multi-attach|failedattach|volume"
kubectl get volumeattachment | grep <pv-name>
kubectl get node # is the old node NotReady / gone?
```

ROOT CAUSES

The failed node is NotReady, so Kubernetes waits before force-detaching the RWO volume.

Some CSI drivers taint/associate the PVC with the old node's identity, blocking rescheduling (a documented pattern on spot instances).

A stale VolumeAttachment object referencing the dead node.

Resolution

Let the controller's force-detach timeout elapse, or remove the dead node/stale attachment so the volume can move.

```
# If the node is truly gone, deleting it lets the attach-detach controller release RWO
kubectl delete node <dead-node>

# As a last resort, remove the stale VolumeAttachment (verify the node is dead first!)
kubectl delete volumeattachment <attachment-name>
```

Fig C5 — only remove attachments/nodes you have confirmed are genuinely dead.

PREVENTION

Prefer topology-aware, faster-detaching CSI drivers for DB workloads.

Avoid spot/preemptible nodes for stateful primaries, or accept the detach latency.

Alert on pods in ContainerCreating > a few minutes.

Scenario C6 Orphaned PV / reclaim-policy data-loss trap

Symptoms

- After deleting/recreating a cluster, old PVs linger as Released, or data is gone
- PV reclaim policy is Delete and the volume vanished with the PVC
- Migrations from Crunchy PGO where PV reuse was intended but data was deleted

Diagnosis

The PV reclaim policy decides what happens to data when a PVC is deleted. Delete means the backing volume is destroyed. Confirm the policy before any delete.

```
kubectl get pv -o custom-columns=\
NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy,\
STATUS:.status.phase,CLAIM:.spec.claimRef.name | grep -i cluster1
```

Fig C6 — check reclaim policy BEFORE deleting any PVC you might need to reuse.

ROOT CAUSES

Default **Delete** reclaim policy destroyed the underlying volume when the PVC went.

A cluster delete removed PVCs that had valuable data with no backup to fall back on.

PV/PVC label mismatch left a Released PV that won't rebind.

Resolution

If the data still exists, set Retain and rebind. If the volume was deleted under a Delete policy, the data is gone — restore from pgBackRest.

```
# Protect a volume you intend to reuse BEFORE deleting its PVC
kubectl patch pv <pv-name> -p \
'{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'

# If the volume is already gone, there is no PV recovery — restore from backup (D5)
```

Fig C6b — Retain is a decision you make in advance, not a recovery step.

PREVENTION

Set **Retain** on production DB PVs (or use a StorageClass that does).

Never rely on PVs as your backup — pgBackRest is the recovery path.

Snapshot volumes before destructive cluster operations.

D. pgBackRest & Backups

pgBackRest runs as a dedicated **repo-host** pod plus a sidecar in each database pod. The primary's **archive_command** pushes WAL to the repo; scheduled full and differential backups run as Jobs. The failure that hurts most is not a failed backup — it's failed *archiving*, because unarchived WAL cannot be recycled and the disk fills (the cascade in D2). Treat archiving health as a first-class SLO.

Scenario D1 Stanza not created / config hash mismatch

Symptoms

- Operator events: **UnableToCreateStanzas**, **StanzaNotCreated**, 'pgBackRest config hash mismatch'
- New cluster never becomes Ready; backups can't run because there's no stanza
- Single-primary (no replica) clusters historically hit stanza-create edge cases

Diagnosis

The stanza is pgBackRest's per-cluster metadata. Read the operator + repo logs to see why stanza-create is failing.

```
kubectl get events -n pgo --field-selector reason=UnableToCreateStanzas
kubectl logs deploy/percona-postgresql-operator -n pgo | grep -i stanza
kubectl logs cluster1-repo-host-0 -n pgo -c pgbackrest --tail=100
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest info
```

ROOT CAUSES

A config hash mismatch: the pgBackRest config the Operator rendered doesn't match what the repo pod holds (often mid-reconcile or after a hand edit).

Duplicate/invalid repo options in the rendered config (e.g. both repo1-path and repo-path).

Repo storage (PVC or object store credentials) not reachable when stanza-create ran.

S3/GCS/Azure secret wrong — bucket/keys/endpoint mismatch.

Resolution

Fix the config or credentials, then let the Operator retry — or trigger stanza-create explicitly once the repo is reachable.

```
# Verify object-store creds / repo PVC are actually usable from the repo pod
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db stanza-create --log-level-console=info

# If it was a transient hash mismatch, a reconcile nudge often clears it
kubectl annotate pg cluster1 -n pgo pgv2.percona.com/trigger-reconcile="$(date +%s)"
--overwrite
```

Fig D1 — confirm repo reachability first; stanza-create fails silently if the repo isn't there.

PREVENTION

Validate backup secrets (object-store keys, endpoints) before first apply.

Don't hand-edit the pgBackRest ConfigMap on a managed cluster (hash mismatch).

For single-node test clusters, confirm your Operator version's stanza behaviour.

Scenario D2 WAL archiving failing — the pg_wal fill cascade

Symptoms

- Postgres log: 'archive command failed', repeated 'archiving ... failed too many times'
- `pg_stat_archiver.failed_count` climbing; `.ready` files piling up
- Ends in a full WAL volume (C2) and a halted primary if not caught

Diagnosis

This is the single most important operational failure to catch early. WAL that can't be archived can't be recycled; `pg_wal` grows until the volume is full and the primary stops.

```
# Archiver health — the earliest signal
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \
"SELECT archived_count, failed_count, last_archived_wal,
last_failed_wal, last_failed_time FROM pg_stat_archiver;"

# The exact failing command + reason from the PG log
kubectl logs cluster1-instance1-abcd-0 -n pgo -c database | \
grep -iE "archive command failed|archive-push|pgbackrest" | tail

# Backlog size — how close to disaster?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
bash -c 'ls -l /pgdata/*/pg_wal/archive_status/*.ready | wc -l'
```

Fig D2 — `failed_count` rising + growing `.ready` backlog = the cascade has started.

ROOT CAUSES

Repo-host pod down or unreachable, so archive-push has nowhere to write (see D6).

Stanza/archive.info missing or unreadable — 'unable to load info file archive.info' (often a repo-path or permissions problem).

Repo storage full (D4) — pushes fail with no space.

A duplicate WAL with a different checksum already in the archive (e.g. after an improper restore onto the same repo) blocking that segment.

Object-store throttling/credentials expiry.

Resolution

Restore the archive path fast — every minute of failed archiving is WAL you can't recycle. Fix the repo, confirm a manual push succeeds, watch the backlog drain.

```
# 1. Prove the repo path works and the stanza is valid
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db check

# 2. If archive.info is missing but backups exist, do NOT wipe the repo — repair the
# stanza (or fix repo-path/creds). If the repo is genuinely new/empty:
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db stanza-create --log-level-console=info

# 3. Force-drain a test segment and confirm failed_count stops rising
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c "SELECT
pg_switch_wal();"
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \
"SELECT failed_count,last_archived_wal FROM pg_stat_archiver;"
```

Fig D2b — a duplicate-checksum block usually means a repo was reused across restores; resolve deliberately, don't blind-delete.

PREVENTION

Make `pg_stat_archiver.failed_count` a paging alert — this is your smoke detector.

Alert on WAL **archive backlog** (.ready count) and on repo-host pod down.

Keep WAL on a separate PVC so the cascade has room before it becomes an outage.

Never reuse one repo across independent restore lineages without a fresh stanza.

Scenario D3 Full / delta backup job stuck or failing

Symptoms

- Scheduled backup Job stuck in 'starting' or ends Failed; PerconaPGBBackup CR not Ready
- Manual backup annotation/CR seems ignored
- Delta backup errors referencing a missing prior full backup

Diagnosis

Backups run as Jobs driven by CronJobs (schedules) or a PerconaPGBBackup CR. Check the CR status, then the Job/pod logs.

```
kubectl get pg-backup -n pgo
kubectl describe pg-backup <backup-name> -n pgo | grep -A8 -iE "status|state|message"
kubectl get jobs -n pgo -l postgres-operator.crunchydata.com/pgbackrest-backup
kubectl logs job/<backup-job> -n pgo --tail=100
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest info
```

ROOT CAUSES

A prior backup Job is still running — the Operator won't start another (and this also blocks pausing the cluster).

Delta/differential requested with no valid full backup to base on.

Repo unreachable or full (D4).

Stanza broken (D1) so the backup can't proceed.

Backup config hash mismatch mid-reconcile.

Resolution

Clear the stuck Job, ensure a valid full exists, then re-trigger. Take a manual full backup via a PerconaPGBBackup CR.

```
# Remove a stuck/old backup job so a new one can start
kubectl delete job -l postgres-operator.crunchydata.com/pgbackrest-backup -n pgo

# Trigger a manual FULL backup via CR (delta needs a full to exist first)
cat <<'YAML' | kubectl apply -f -
apiVersion: pgv2.percona.com/v2
kind: PerconaPGBBackup
metadata: {name: cluster1-full-ondemand, namespace: pgo}
spec: {pgCluster: cluster1, repoName: repo1, options: ["--type=full"]}
YAML

kubectl get pg-backup cluster1-full-ondemand -n pgo -w
```

Fig D3 — always ensure a valid full exists before relying on delta/diff.

PREVENTION

Alert on backup Job failures and on **time-since-last-successful-full/diff**.

Verify the full/diff schedule cadence matches your retention and RPO.

Periodically run `pgbackrest info` in a check job.

Scenario D4 Repo storage full / retention not pruning

Symptoms

- Repo PVC or bucket at capacity; new backups and archive-push fail
- Old backups never expiring; backup count exceeds intended retention
- This silently becomes D2 (archiving fails) once the repo is full

Diagnosis

Retention (full/diff/archive) controls pruning. If it's misconfigured or expiry never runs, the repo grows forever.

```
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest info
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- df -h /pgbackrest
# Review configured retention (rendered from the CR)
kubectl get pg cluster1 -n pgo -o jsonpath='{.spec.backups.pgbackrest.repos}' | tr ',' '\n'
```

ROOT CAUSES

repo retention (repo1-retention-full / -diff / -archive) unset or too generous.

Expiry tied to full-backup retention: if fulls aren't being taken, archives aren't pruned either.

Object-store lifecycle not configured for the archive prefix.

A stuck backup leaving partial artifacts consuming space.

Resolution

Set correct retention so expiry prunes automatically; reclaim space, then confirm archiving resumes.

```
# Set retention via the CR (Operator renders pgBackRest options)
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"backups": {"pgbackrest": {"global": {
"repo1-retention-full": "7",
"repo1-retention-diff": "14",
"repo1-retention-archive": "14"}}}}}'

# Manual expire to reclaim now (retention drives what is removed)
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- \
pgbackrest --stanza=db expire --log-level-console=info
```

Fig D4 — archive retention prunes only alongside the fulls it protects; set both.

PREVENTION

Alert on repo utilisation with headroom for at least one full + its WAL.

Size retention to your RPO/PITR window, not 'forever'.

For object stores, add a lifecycle policy as a backstop to pgBackRest expiry.

Scenario D5 Restore & PITR failures

Symptoms

- Restore Job errors; cluster won't come up on restored data
- 'WAL segment ... was not archived before timeout' during recovery
- Ownership warnings ('unknown group postgres in backup manifest'), or PITR target not reached

Diagnosis

Restores run via a PerconaPGRestore CR (or pgDataSource for a new cluster). Read the restore Job logs and confirm the target backup + WAL exist.

```
kubectl get pg-restore -n pgo
kubectl logs job/<restore-job> -n pgo --tail=150
# Confirm the backup set and WAL range you're targeting actually exist
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest info
```

ROOT CAUSES

Target WAL for the PITR point isn't in the archive (retention pruned it, or archiving had a gap — see D2).

Restoring onto a repo/timeline that diverged, causing checksum/timeline conflicts.

The 'unknown group postgres' warning is benign (uid/gid mapping) — not the real error; look past it for the true failure.

PITR target (time/LSN/name) specified incorrectly or unreachable.

Resolution

Restore to the nearest valid point with WAL coverage; for a new-cluster restore use `pgDataSource`, for in-place use `PerconaPGRestore` (destructive).

```
# Point-in-time restore in place (DESTRUCTIVE to the existing cluster data)
cat <<'YAML' | kubectl apply -f -
apiVersion: pgv2.percona.com/v2
kind: PerconaPGRestore
metadata: {name: cluster1-pitr, namespace: pgo}
spec:
  pgCluster: cluster1
  repoName: repo1
  options:
    - --type=time
    - "--target=2026-07-28 09:30:00+00"
YAML
kubectl get pg-restore cluster1-pitr -n pgo -w
```

Fig D5 — verify WAL coverage for your target BEFORE starting a PITR; retention must span it.

PREVENTION

Rehearse restores regularly — an untested backup is a hypothesis, not a backup.

Keep archive retention long enough to cover your full PITR window.

Document exact restore CRs for new-cluster vs in-place so on-call isn't improvising.

Scenario D6 Repo-host pod down — archive-push failing fleet-wide

Symptoms

- Repo-host pod CrashLoop/Pending; every cluster using it can't archive or back up
- archive-push failures across all databases pointing at that repo
- Backups queue up; WAL backlog grows everywhere (D2 fleet-wide)

Diagnosis

The repo-host is a single choke point for archiving and backup. Treat its outage as an archiving incident, not just 'a pod is down'.

```
kubectl get pod cluster1-repo-host-0 -n pgo -o wide
kubectl describe pod cluster1-repo-host-0 -n pgo | grep -A10 Events
kubectl logs cluster1-repo-host-0 -n pgo -c pgbackrest --tail=100
# Is its PVC bound / full? (repo-host often has its own repo PVC)
kubectl get pvc -n pgo | grep repo
```

ROOT CAUSES

Repo PVC full (D4), Pending (C3), or won't attach (C5).

Repo-host pod evicted/OOM or scheduled to a node without the repo volume's topology.

Object-store credentials/secret invalid so the pod can't initialise.

Operator mid-reconcile after a backup config change.

Resolution

Get the repo-host healthy first (storage or scheduling), then confirm archiving drains across all dependent clusters.

```
# Common cause is storage – resolve C3/C4/D4 for the repo PVC, then:
kubectl delete pod cluster1-repo-host-0 -n pgo # StatefulSet recreates it
kubectl exec -it cluster1-repo-host-0 -n pgo -c pgbackrest -- pgbackrest --stanza=db check

# Then verify each database's archiver has recovered
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -tAc \
"SELECT failed_count,last_archived_time FROM pg_stat_archiver;"
```

Fig D6 — one repo-host outage can back up WAL on every cluster that archives to it.

PREVENTION

Alert on repo-host pod availability as a high-severity, fleet-level signal.

Give the repo-host resilient, monitored storage with expansion enabled.

Consider multiple repos (e.g. local PVC + object store) so archiving has a fallback.

E. Pods, Operator & Lifecycle

These are the Kubernetes-native failure modes: scheduling, crash loops, memory, and the Operator's own reconcile loop. The recurring lesson is to read the CR status and the Operator log *together* — the Operator often knows exactly why it is stuck, and will tell you.

Scenario E1 Cluster stuck 'initializing' / pods Pending

Symptoms

- CR status stays initializing for a long time
- One or more pods Pending / unschedulable
- New cluster never reaches Ready

Diagnosis

Initializing-for-too-long specifically means 'go read the Operator logs' — and a Pending pod means scheduling, not Postgres.

```
kubectl get pg cluster1 -n pgo -o yaml | grep -A20 "status:"
kubectl get pods -n pgo -o wide | grep -iE "pending|containercreating|init"
kubectl describe pod <pending-pod> -n pgo | grep -A12 Events
kubectl logs deploy/percona-postgresql-operator -n pgo --tail=150
```

ROOT CAUSES

Unschedulable pod: insufficient CPU/memory, taints without tolerations, or affinity that no node satisfies.

PVC Pending (C3) blocking the pod.

Stanza-create failing keeps the cluster from completing init (D1).

Image pull failure (registry auth / wrong tag).

Resolution

Resolve the specific blocker the events name — capacity, affinity, PVC, or image.

```
kubectl describe nodes | grep -A5 -iE "allocated resources|taints"
kubectl get events -n pgo --sort-by=.lastTimestamp | tail -20
# e.g. relax anti-affinity for a small cluster, or add capacity/tolerations
```

Fig E1 — 'initializing' is the Operator pointing you at scheduling/PVC/stanza, not at Postgres.

PREVENTION

Right-size requests and validate anti-affinity against real node count.

Pre-flight image tags and registry secrets before applying.

Watch CR conditions; alert when Progressing persists too long.

Scenario E2 Database pod CrashLoopBackOff

Symptoms

- Database container restarts in a loop; Restart Count climbing

- Logs show FATAL/PANIC at startup, or a specific parameter rejected
- May follow a bad config change, an unclean shutdown, or storage trouble

Diagnosis

Read the last termination reason and the startup logs — Postgres tells you precisely why it won't start.

```
kubectl logs <pod> -n pgo -c database --previous | tail -60
kubectl describe pod <pod> -n pgo | grep -A4 -iE "last state|reason|exit code"
```

ROOT CAUSES

Invalid parameter (e.g. a value that needs a restart set wrong, or too high for the container's memory).

Unclean shutdown -> Postgres doing crash recovery (let it finish; watch progress).

Corrupt/partial data dir after a storage incident (C-series).

Full volume (C1/C2).

pg_hba / auth config making the readiness probe fail so K8s keeps killing it.

Resolution

Fix the specific startup error. If it's config, take the cluster unmanaged and correct via patronictl; if it's data, restore.

```
# If a bad Patroni-managed parameter blocks startup:
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":true}}'
kubectl exec -it <pod> -n pgo -c database -- patronictl edit-config \
--set postgresql.parameters.<param>=<sane-value>
# after it starts cleanly:
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":false}}'

# If it's crash recovery, DON'T kill it — watch it converge:
kubectl logs <pod> -n pgo -c database -f | grep -iE "redo|recovery|consistent"
```

Fig E2 — distinguish 'bad config' (fix it) from 'crash recovery' (let it run).

PREVENTION

Stage parameter changes in non-prod; know which need a restart.

Alert on restart count, not just readiness, to catch a slow crash loop.

Keep tested backups so a corrupt data dir is a restore, not a disaster.

Scenario E3 OOMKilled — memory sizing

Symptoms

- Database container Last State = OOMKilled; periodic restarts under load
- Correlates with heavy queries, large sorts, or many connections
- Often presents upstream as failover flapping (A3)

Diagnosis

PostgreSQL memory is $\text{shared_buffers} + \text{per-connection work_mem} \times \text{concurrency} + \text{maintenance}$. If limits ignore that math, the kernel kills it.

```
kubectl describe pod <pod> -n pgo | grep -iE "oomkilled|memory"
kubectl exec -it <pod> -n pgo -c database -- psql -c \
"SHOW shared_buffers; SHOW work_mem; SHOW max_connections;"
kubectl top pod <pod> -n pgo
```

ROOT CAUSES

Container memory limit lower than $\text{shared_buffers} + \text{realistic work_mem} \times \text{active connections}$.

work_mem too high combined with many concurrent sorts/hashtables.

A connection spike (missing pooling — see F1) multiplying per-backend memory.

$\text{maintenance_work_mem} \times \text{parallel maintenance during large index builds/VACUUM}$.

Resolution

Size the limit to the workload and cap per-connection memory; put a pooler in front to bound concurrency.

```
# Raise the limit AND keep request==limit (Guaranteed QoS)
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"instances": [{"name": "instance1",
"resources": {"requests": {"memory": "16Gi"}, "limits": {"memory": "16Gi"}}}]}'

# And bound per-connection memory via Patroni-managed params
kubectl exec -it <pod> -n pgo -c database -- patronictl edit-config \
--set postgresql.parameters.work_mem=16MB
```

Fig E3 — total memory scales with concurrency; cap work_mem and pool connections.

PREVENTION

Budget memory: $\text{shared_buffers} \sim 25\%$ of limit, plus $\text{work_mem} \times \text{expected active queries}$.

Front the DB with PgBouncer to bound backend count (F1).

Alert on OOMKilled events explicitly.

Scenario E4 Operator reconcile errors / stuck reconciliation

Symptoms

- Operator log: 'Reconciler error' repeating for the CR
- CR changes don't take effect; status doesn't advance
- Sometimes 'failed to get operator pod/deployment' in cluster-wide installs

Diagnosis

The Operator can't complete a reconcile. The log line names the failing step — RBAC, a missing dependency, an invalid CR field, or a webhook.

```
kubectl logs deploy/percona-postgresql-operator -n pgo --tail=200 | grep -iE
"error|reconcile"
kubectl get pg cluster1 -n pgo -o yaml | grep -A20 "status:"
# Cluster-wide mode: does the operator have RBAC to see its own deployment/target ns?
kubectl auth can-i --list --as=system:serviceaccount:pgo:percona-postgresql-operator -n pgo
```

ROOT CAUSES

RBAC gaps in cluster-wide/watch-namespace installs (operator can't read required objects).

Invalid CR field or an unsupported combination the Operator rejects.

A dependent object stuck (PVC/Job) that reconcile keeps waiting on.

Version skew between CRDs and the Operator image after a partial upgrade.

Resolution

Fix the named cause — grant RBAC, correct the CR, or unblock the dependency. For a wedged reconcile, restarting the Operator pod is safe and often clears transient state.

```
# Safe: bounce the operator to clear a wedged in-memory reconcile
kubectl rollout restart deploy/percona-postgresql-operator -n pgo

# Ensure CRDs match the operator version (partial upgrades cause skew)
kubectl get crd | grep percona
kubectl get pg cluster1 -n pgo -o jsonpath='{.apiVersion}{"\n"}'
```

Fig E4 — restarting the Operator is non-destructive; the databases keep running.

PREVENTION

Keep CRDs and Operator image in lockstep during upgrades.

Verify RBAC/watch-namespace config for cluster-wide installs.

Alert on sustained reconcile errors; they precede visible outages.

Scenario E5 Rolling (smart) update stuck mid-rollout

Symptoms

- An image/config change starts a rolling restart that never finishes
- Some pods on the new spec, some old; primary last but not switching
- CR parked in Progressing

Diagnosis

The Operator restarts replicas first, then switches the primary last. A stuck rollout is usually one pod that won't become Ready, blocking the sequence.

```
kubectl get pods -n pgo -o wide -l postgres-operator.crunchydata.com/cluster=cluster1
kubectl describe pod <not-ready-pod> -n pgo | grep -A10 Events
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list
kubectl logs deploy/percona-postgresql-operator -n pgo --tail=120 | grep -iE
"update|restart|switchover"
```

ROOT CAUSES

One replica won't come Ready (resources, PVC, bad new config), so the rollout can't proceed to the primary.

The primary switchover step is hanging (A4).

A new parameter value in the update is invalid and the restarted pod crash-loops (E2).

Resolution

Unblock the offending pod (or roll the change back), then let the sequence complete; take the cluster unmanaged only if you must intervene by hand.

```
# Roll back the triggering change if it's the cause
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"previous good value... }}'

# Or intervene manually on a stuck switchover
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":true}}'
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
patronictl switchover cluster1-ha --candidate <ready-replica> --force
kubectl patch pg cluster1 -n pgo --type merge -p '{"spec":{"unmanaged":false}}'
```

Fig E5 — a stuck rollout is usually one un-Ready pod; fix that and the sequence resumes.

PREVENTION

Apply risky changes in a maintenance window; validate new values in non-prod first.

Don't stack multiple spec changes into one apply during business hours.

Monitor Progressing-condition duration.

Scenario E6 Config change silently reverted by the Operator

Symptoms

- You changed a PG parameter or edited a ConfigMap and it reverted within seconds
- patronictl edit-config changes disappear on the next reconcile
- 'Why won't my setting stick?'

Diagnosis

On a managed cluster the Operator is the source of truth and reconciles owned objects back to the CR. Your out-of-band edit is expected to be reverted.

```
# Confirm the cluster is managed (unmanaged:false or unset) – that's why it reverts
kubectl get pg cluster1 -n pgo -o jsonpath='{.spec.unmanaged}'
```

ROOT CAUSES

The change was made outside the CR on a managed cluster, so reconcile reverts it.

You edited a ConfigMap/StatefulSet the Operator owns.

Two sources of truth: CR vs manual patronictl.

Resolution

Make durable changes through the CR. Use unmanaged mode (or the documented Patroni annotation) only for temporary, deliberate overrides.

```
# Durable: set the parameter in the CR so the Operator keeps it
kubectl patch pg cluster1 -n pgo --type merge -p '{
"spec": {"patroni": {"dynamicConfiguration": {"postgresql": {"parameters": {
"log_min_duration_statement": "500ms"}}}}}}}'

# Temporary manual override: unmanaged first, then patronictl; revert when done.
```

Fig E6 — the CR is the source of truth on a managed cluster; edit there for changes that must persist.

PREVENTION

Treat the CR as the single source of truth; keep it in Git (you already run ArgoCD).

Reserve unmanaged mode for short, deliberate maintenance only.

Document that manual edits on a managed cluster are transient by design.

F. Connections & PgBouncer

The Operator can front PostgreSQL with a **PgBouncer** pooler. Connection problems split into three shapes: too many backends reaching Postgres, the pooler itself failing, and traffic landing on the wrong role after a failover.

Scenario F1 Too many connections / pool exhaustion

Symptoms

- 'FATAL: sorting connection ... too many clients already' / remaining connection slots reserved
- Apps intermittently can't connect; backend count near max_connections
- Correlates with OOM (E3) as each backend costs memory

Diagnosis

Decide whether the fix is more slots (rarely) or fewer backends via pooling (usually).

```
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \  
"SELECT count(*), state FROM pg_stat_activity GROUP BY state;"  
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c "SHOW  
max_connections;"  
# Pooler view  
kubectl exec -it cluster1-pgbouncer-abcd -n pgo -- \  
psql -p 5432 pgbouncer -c "SHOW POOLS;" 2>/dev/null
```

ROOT CAUSES

Application opening far more connections than the DB should hold (no pooling / pool per replica).

Idle-in-transaction sessions holding slots.

max_connections raised to paper over the problem (raising per-backend memory and OOM risk — and remember the standby-first ordering, B2).

Monitoring/exporter itself opening too many connections.

Resolution

Put PgBouncer in transaction mode in front of the DB and size pools sensibly; kill idle-in-transaction offenders.

```
# Enable/scale the pooler via the CR  
kubectl patch pg cluster1 -n pgo --type merge -p '{  
  "spec": {"proxy": {"pgBouncer": {"replicas": 3,  
    "config": {"global": {"pool_mode": "transaction", "default_pool_size": "50"}}}}}'  
  
# Reap idle-in-transaction sessions older than 10 min (investigate the app too)  
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- psql -c \  
"SELECT pg_terminate_backend(pid) FROM pg_stat_activity  
WHERE state='idle in transaction' AND now()-state_change > interval '10 min';"
```

Fig F1 — pool in transaction mode to bound backends instead of inflating max_connections.

PREVENTION

Route apps through PgBouncer; don't let services connect straight to Postgres.

Set `idle_in_transaction_session_timeout` to auto-reap leaks.

Cap the exporter's connections (PMM/postgres_exporter supports a connection limit).

Scenario F2 PgBouncer pod failing or auth errors

Symptoms

- Apps get auth failures through the pooler though direct DB auth works
- PgBouncer pod CrashLoop or not Ready
- 'no such user' / password auth failed at the pooler layer

Diagnosis

The pooler has its own auth to the DB and its own config. Separate a pooler problem from a database-auth problem.

```
kubectl get pod -l postgres-operator.crunchydata.com/role=pgbouncer -n pgo
kubectl logs <pgbouncer-pod> -n pgo | tail -50
# Does the same credential work directly against the DB?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- \
psql -U <appuser> -c "SELECT 1;"
```

ROOT CAUSES

Pooler auth secret out of sync with the DB user's password (rotation without pooler refresh).

Pool user missing required privileges / not in the pooler's auth query.

Pooler config change mid-reconcile.

Pooler pod resource-starved.

Resolution

Re-sync the pooler credentials via the CR-managed secret and restart the pooler; verify end-to-end through the pooler.

```
# Let the Operator reconcile pooler auth, then bounce the pooler
kubectl rollout restart deploy -l postgres-operator.crunchydata.com/role=pgbouncer -n pgo
# End-to-end test THROUGH the pooler service
kubectl run pgtest --rm -it --image=percona/percona-distribution-postgresql:16 -n pgo -- \
psql "host=cluster1-pgbouncer port=5432 user=<appuser> dbname=<db>" -c "SELECT 1;"
```

Fig F2 — always verify through the pooler service, not just direct to the DB.

PREVENTION

Automate credential rotation so pooler and DB never drift.

Alert on pooler pod readiness and on auth-failure rate at the pooler.

Give the pooler its own modest resource requests/limits.

Scenario F3 App writes hitting a replica after failover (stale routing)

Symptoms

- Post-failover, writes fail with 'cannot execute INSERT in a read-only transaction'
- App still connected to the old primary (now a replica) or the -ha service is stale
- Reads succeed, writes don't

Diagnosis

Writes must go to the **-ha** (primary) service, reads to **-replicas**. After a failover, confirm the service endpoints followed the new primary.

```
# Who does Patroni say is primary?
kubectl exec -it cluster1-instance1-abcd-0 -n pgo -c database -- patronictl list
# Does the -ha service endpoint point at that pod?
kubectl get endpoints cluster1-ha -n pgo -o wide
kubectl get pods -n pgo -L postgres-operator.crunchydata.com/role
```

ROOT CAUSES

The app pins a pod IP or caches connections instead of using the -ha service DNS.

Endpoint/label update lagged the promotion briefly.

App connected to the read-only -replicas service for writes by mistake.

Resolution

Point writes at the -ha service and make the app reconnect; the primary label/endpoint follows Patroni automatically once state settles.

```
# Confirm the -ha service now targets the new primary, then force app reconnect
kubectl get svc cluster1-ha -n pgo
kubectl rollout restart deploy/<app> -n <app-ns> # drop stale cached connections
```

Fig F3 — write to -ha, read from -replicas; never pin pod IPs.

PREVENTION

Use the **-ha** service DNS for writes and **-replicas** for reads — never pod IPs.

Set client keepalives/short connection lifetimes so failover reconnects quickly.

Test app behaviour during a game-day failover, not just the DB layer.

G. Monitoring, Alerting & CronJobs

Your stack is self-hosted Prometheus, Grafana and Alertmanager, with backups and checks driven by CronJobs. The failure modes here are quieter and more dangerous — a broken exporter or a suspended CronJob means you lose visibility exactly when you need it. The recurring theme: **monitor the monitoring**.

Scenario G1 Exporter down / no PostgreSQL metrics

Symptoms

- Grafana panels blank; PostgreSQL metrics absent from Prometheus
- Exporter/sidecar (postgres_exporter or PMM client) not Ready
- Prometheus target for the DB shows DOWN

Diagnosis

Find where the chain breaks: exporter running -> exposing /metrics -> Prometheus scraping -> query permissions.

```
# Is the exporter serving metrics at all?
kubectl exec -it <db-pod> -n pgo -c <exporter-container> -- \
curl -s localhost:9187/metrics | head
# Does the monitoring user have the rights it needs?
kubectl exec -it <db-pod> -n pgo -c database -- psql -c "\du" | grep -i monitor
```

ROOT CAUSES

Monitoring DB user lacks privileges (needs to read pg_stat_* views; PMM recommends a dedicated account with sufficient rights).

Exporter container crashed or misconfigured DSN.

Too many databases: auto-discovery has a default limit (e.g. 10) — extra DBs silently unmonitored.

Exporter opening too many connections and being throttled/blocked (F1 overlap).

Resolution

Restore the exporter's access and config; verify Prometheus can scrape it.

```
# Restart the exporter/sidecar and re-check the target in Prometheus
kubectl delete pod <db-pod> -n pgo # sidecar restarts with the pod (replica first!)
# In Prometheus: Status > Targets — the DB target should return to UP
# If many DBs, raise/disable the exporter's auto-discovery database limit
```

Fig G1 — a low auto-discovery limit silently drops metrics for databases beyond the cap.

PREVENTION

Alert on the exporter target being **down** (a dead exporter can't alert on itself).

Grant the monitoring user exactly the privileges the exporter needs, no more.

Set the auto-discovery DB limit to match your real database count.

Scenario G2 Prometheus target flaps after every failover

Symptoms

- Targets churn on each failover; gaps in metrics around role changes
- Duplicated or missing series as pods are relabelled
- Alerts fire/resolve spuriously during rollouts

Diagnosis

Pod churn (failover, rollout, reschedule) changes pod names/IPs. If scraping keys off pod identity rather than stable service/role labels, targets flap.

```
# How is Prometheus discovering the DB pods? (service vs pod SD, relabeling)
kubectl get servicemonitor,podmonitor -n pgo 2>/dev/null
kubectl get pods -n pgo -L postgres-operator.crunchydata.com/role # role label follows Patroni
```

ROOT CAUSES

Scrape config targets ephemeral pod IPs instead of the stable headless service.

Relabeling doesn't carry the Operator's role label (master/replica), so series aren't stable across promotions.

Rollouts recreate pods faster than scrape/stale handling expects.

Resolution

Discover via the headless service and keep the role label; use it to distinguish primary vs replica series consistently.

```
# Prefer service/endpoints discovery with the operator's role label preserved.
# Relabel to keep a stable 'role' and 'cluster' label so series survive failover:
# source_labels: [__meta_kubernetes_pod_label_postgres_operator_crunchydata_com_role]
# target_label: role
```

Fig G2 — key metrics off stable cluster/role labels, not pod name/IP, so failover doesn't create gaps.

PREVENTION

Base alert rules on role labels (alert on 'no primary' rather than 'pod X down').

Allow for brief expected gaps during rollouts to avoid alert noise.

Test dashboards/alerts through a game-day failover.

Scenario G3 Alertmanager not firing (or firing everything)

Symptoms

- A real incident produced no page; or trivial blips page constantly
- Alertmanager silences swallowing real alerts; routing/receiver misconfigured
- Inhibition rules hiding the alert that matters

Diagnosis

Separate 'rule didn't fire' from 'fired but didn't notify'. Check the rule in Prometheus, then the route/silences in Alertmanager.

```
# Did the alert rule evaluate/fire in Prometheus? (Status > Rules / Alerts)
# Then in Alertmanager: active alerts, silences, and the route tree
kubectl exec -it <alertmanager-pod> -n monitoring -- \
amtool alert query 2>/dev/null
kubectl exec -it <alertmanager-pod> -n monitoring -- \
amtool silence query 2>/dev/null
```

ROOT CAUSES

An over-broad silence is suppressing the alert.

Route/receiver misconfig (wrong matcher, missing receiver) so it fires but never notifies.

Inhibition rule hiding a symptom behind a broader alert.

The alert expression itself is wrong (never true, or always true).

Resolution

Fix the specific link: expire the errant silence, correct the route, or fix the expression. Verify end-to-end with a test alert.

```
# Expire a bad silence
kubectl exec -it <alertmanager-pod> -n monitoring -- amtool silence expire <silence-id>
# Validate routing for a given label set
kubectl exec -it <alertmanager-pod> -n monitoring -- \
amtool config routes test --tree severity=critical
```

Fig G3 — 'no page' is usually a silence or route problem, not a missing rule.

PREVENTION

Add a permanent low-severity **watchdog/heartbeat** alert that must always fire — if it stops, your alerting pipeline is broken.

Review silences regularly; prefer time-boxed silences.

Test critical routes after any Alertmanager change.

Scenario G4 Backup / monitoring CronJobs silently not running

Symptoms

- Expected scheduled backups/checks didn't happen; no Job created
- CronJob shows SUSPEND=True or LAST SCHEDULE long ago
- Jobs fail immediately on RBAC or exec into a non-existent pod name

Diagnosis

CronJob-driven tasks fail quietly — nobody notices until the artifact they produce is missing. Inspect the CronJob and its recent Jobs.

```
kubectl get cronjob -n pgo -o wide # SUSPEND, LAST SCHEDULE, ACTIVE
kubectl get jobs -n pgo --sort-by=.metadata.creationTimestamp | tail
kubectl logs job/<recent-cronjob-run> -n pgo --tail=80
kubectl describe cronjob <name> -n pgo | grep -A6 Events
```

ROOT CAUSES

CronJob left **suspend: true** after maintenance.

The Job's ServiceAccount lacks RBAC to exec into pods / run pgBackRest.

The script targets a hard-coded pod name that changed after a failover/rollout (should target the primary via label/service).

startingDeadlineSeconds / concurrencyPolicy caused missed runs to be skipped.

Node pressure or image pull failure on the Job pod.

Resolution

Un-suspend, fix RBAC, and make the script resolve the primary dynamically by label rather than a fixed pod name.

```
# Un-suspend a CronJob left off after maintenance
kubectl patch cronjob <name> -n pgo -p '{"spec":{"suspend":false}}'

# Resolve the CURRENT primary by label instead of hard-coding a pod name
PRIMARY=$(kubectl get pods -n pgo \
-l postgres-operator.crunchydata.com/role=master \
-o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$PRIMARY" -n pgo -c database -- psql -c "SELECT 1;"
```

Fig G4 — never hard-code a pod name in a scheduled job; the primary moves on failover.

PREVENTION

Alert on **time-since-last-successful-run** for every critical CronJob, and on CronJob suspend=true.

Have jobs target the primary by role label / -ha service, never a fixed pod name.

Give job ServiceAccounts least-privilege RBAC and test it.

Emit a success metric (pushgateway/textfile) so a missing run is itself alertable.

Appendix — First-response cheat sheet

Pin this. When the pager goes off, this maps a symptom to the domain and the first command to run.

Symptom	Likely domain	First command
No primary / writes fail	A1, F3	patronictl list
Two primaries / diverging data	A2	patronictl list + compare timeline_id
Pods restarting in a loop	A3, E2, E3	kubectl describe pod (Last State)
Replica lag rising	B1	pg_stat_replication on primary
Replica won't rejoin	A5, B2, B5	patronictl list -e (TL + state)
Disk filling up	C1, C2, B3, D4	df -h /pgdata /pgwal ; pg_replication_slots
Pod Pending	C3, E1	kubectl describe pvc / pod (Events)
Pod ContainerCreating (stuck)	C5	describe pod grep Multi-Attach
Archiving errors / WAL backlog	D2, D6	pg_stat_archiver.failed_count
Backup failed / stuck	D3, D4	kubectl get pg-backup ; pgbackrest info
Restore failing	D5	kubectl logs job/ ; pgbackrest info
CR won't reconcile	E4, E5	operator logs grep -i error
Config change won't stick	E6	check spec.unmanaged
Too many connections	F1	pg_stat_activity group by state
No metrics in Grafana	G1, G2	curl exporter /metrics ; Prometheus targets
Scheduled job didn't run	G4	kubectl get cronjob (SUSPEND / LAST SCHEDULE)

The cascade to watch above all others

THE WAL FILL CASCADE

Archiving fails (D2/D6) or a slot goes dead (B3) → WAL cannot be recycled → pg_wal fills (C2) → the primary halts. Every stage has an early signal — **pg_stat_archiver.failed_count**, an **inactive slot with large retained WAL**, and **WAL PVC utilisation**. Alert on all three and you will catch this hours before it becomes an outage. This one cascade is behind a large share of 'the database suddenly went down' incidents on the Operator.

Key locations & identifiers

Thing	Where
Cluster CR	kind PerconaPGCluster (short: pg), apiVersion pgv2.percona.com/v2
Backup / restore CRs	PerconaPGBackup (pg-backup), PerconaPGRestore (pg-restore)
Instance pods	cluster1-instance1--0 (container: database)
Repo host pod	cluster1-repo-host-0 (container: pgbackrest)
Pooler pods	cluster1-pgbouncer-
Primary service (writes)	cluster1-ha

Thing	Where
Replica service (reads)	cluster1-replicas
Headless service	cluster1-pods
Patroni REST / probes	https://:8008/ (liveness, readiness, health)
Role label	postgres-operator.crunchydata.com/role = master replica
Data / WAL inside pod	/pgdata , /pgwal (if separate WAL volume)

VERSION NOTE

Grounded in the Percona Operator for PostgreSQL v2.x line (current 2.9.x as of mid-2026), which defaults to PostgreSQL 18 and adds PVC-snapshot backups, LDAP, and explicit **StandbyLagging** lag detection; PMM2 monitoring is deprecated in favour of PMM3. CR fields and pod-naming shown match the v2 API — confirm against your deployed version, since field paths shift between minor releases. Behaviour descriptions draw on Percona and Crunchy PGO documentation, Percona community forum incidents, and pgBackRest/Patroni upstream issue trackers.